



**University of
Batangas**
LIPA CITY

Automated License Plate Recognition (ALPR) System for Parking Toll Management

Submitted to:

ENGR. Oscar Bryan Magtibay

Submitted by:

Dominic F. Ramos

CPE4A

December 2025

Abstract

This project presents an Automated License Plate Recognition (ALPR) system designed for parking lots to reduce congestion at toll booths by replacing manual payment with automated kiosks. The system integrates a camera feed, YOLO-based license plate detection, Tesseract OCR, an ESP32 microcontroller with OLED display, and a MongoDB backend. Video frames are processed in real time on a Python host, where license plates are detected and recognized, then displayed on an ESP32 kiosk and logged in a database for analytics and billing. A 10-second cooldown mechanism prevents duplicate entries for the same vehicle. Testing on live vehicle streams demonstrated plate detection and recognition performance suitable for high-throughput environments, along with a payback period on the order of a few months due to reduced staffing needs and increased lane throughput. The results indicate that the proposed ALPR architecture is a feasible and cost-effective alternative to traditional toll booths in modern parking facilities.

I. Introduction

1.1 Background of the Study

Parking management in modern urban environments represents a significant operational challenge. Traditional toll booth systems require manual intervention, creating bottlenecks during peak hours and reducing throughput efficiency. According to recent studies on intelligent transportation systems, automated vehicle identification significantly reduces congestion and improves revenue collection accuracy. License plate recognition technology has emerged as a key enabler for intelligent parking systems, with applications ranging from access control to automated toll collection. The integration of computer vision with IoT devices enables real-time processing at edge locations, reducing latency and infrastructure costs. YOLOv8, a state-of-the-art object detection framework, provides fast and accurate plate detection, while Optical Character Recognition (OCR) engines like Tesseract enable text extraction from detected regions. These technologies, combined with embedded systems like the ESP32 microcontroller, create an accessible and cost-effective solution for parking facility operators.

1.2 Problem Statement

Traditional parking toll collection systems exhibit several limitations:

- Manual toll booths cause traffic congestion due to slow human interaction and payment processing.
- Staffing toll booths create high recurring operational costs.
- Manual recording of transactions is error-prone and difficult to audit.
- Scaling capacity typically requires constructing additional booths and hiring more personnel.

These limitations reduce throughput, increase operating expenses, and degrade user experience. There is a need for a parking toll solution that can automatically identify vehicles, support self-service payment kiosks, and provide accurate digital records without requiring additional manual toll booths.

1.3 Related Literature

Recent work in computer vision shows that deep learning–based detectors (e.g., YOLO-family models) outperform traditional feature-based methods for license plate detection in terms of robustness and speed. OCR engines such as Tesseract have been successfully applied to structured text, including plates, especially when supported by targeted preprocessing and character whitelisting. IoT research demonstrates that microcontrollers like the ESP32 can host lightweight web servers, handle WiFi communication, and drive compact displays, enabling real-time feedback at the edge. Studies in parking management report that automated identification and payment systems can reduce both queue length and staffing costs, while providing data that supports dynamic pricing and occupancy analytics.

The combination of edge computing and cloud connectivity enables hybrid architectures where plate detection occurs locally on a computer or embedded device (reducing bandwidth) while payment processing and data logging occur centrally on a server or cloud platform.

1.4 Objectives

This study aims to:

- Design an integrated ALPR system for parking toll automation using computer vision and IoT components.
- Develop a prototype combining Python-based detection (YOLO, OpenCV, Tesseract) with an ESP32 OLED kiosk and MongoDB backend.
- Test the system on live vehicle streams to evaluate detection accuracy, recognition performance, and end-to-end responsiveness.
- Assess the feasibility of deploying the system in parking lots as an alternative to manual toll booths and fixed-lane operators.

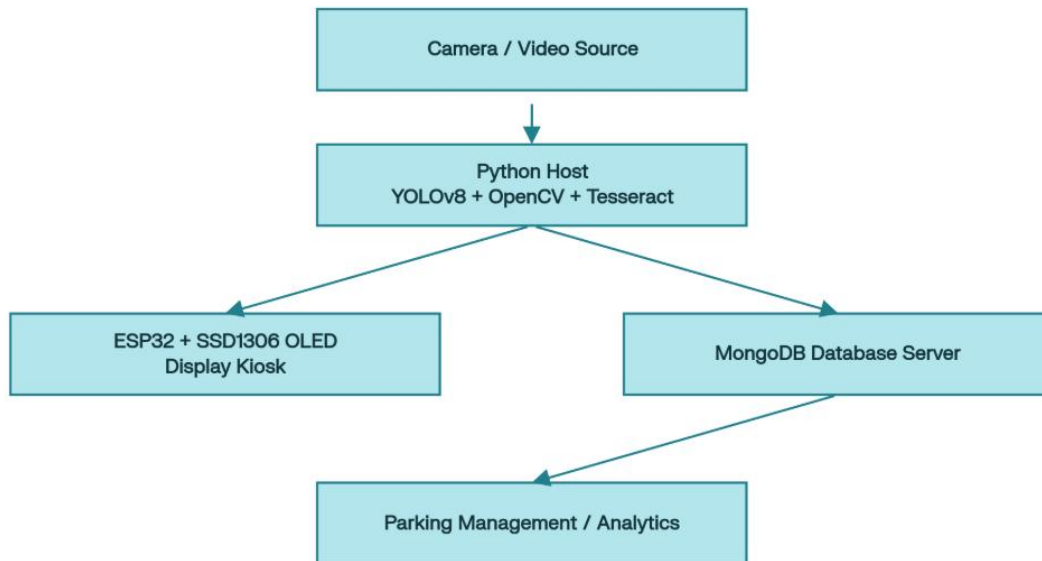
II. Methodology

2.1 System Architecture Diagram

The proposed system consists of three layers working together in real time. A camera provides a continuous video stream of vehicles at the parking exit. A Python host processes each frame, using YOLO to detect license plates and Tesseract to extract text, then applies a cooldown filter to prevent duplicates. Recognized plates are sent via HTTP to an ESP32 microcontroller that drives an SSD1306 OLED display for driver feedback, and are also stored in MongoDB for logging, reporting, and integration with payment systems.



Figure 1: System Architecture Diagram



2.2 Hardware and Software Requirements

Component / Software	Key Specification	Role
IP Camera	1080p, H.264	Vehicle video capture
Host Computer	Python 3.9+, GPU optional	Runs YOLO, OpenCV, Tesseract, API



Component / Software	Key Specification	Role
ESP32 WROOM-32	2.4 GHz WiFi, dual core	Edge kiosk controller
SSD1306 OLED	128×64 px, I2C	Plate display for drivers
MongoDB Server	Local or cloud instance	Persistent data storage
Network Router / Switch	LAN/WiFi	Connects camera, host, ESP32
Python	3.13+	Main implementation language
Ultralytics	8.x	License plate detection model



Component / Software	Key Specification	Role
YOLO		
OpenCV	4.x	Frame capture, preprocessing, display
Tesseract OCR	5.x	Plate text recognition
PyMongo	3.x	Database connectivity
Arduino IDE / PlatformIO	Latest	ESP32 firmware development
Adafruit SSD1306 Library	2.x	OLED driver on ESP32

2.3 Development Approach

An Agile development methodology was used to accommodate iterative model tuning and hardware integration. Initial sprints focused on requirements definition and system architecture design. Subsequent sprints implemented the YOLO-based detection pipeline in Python, integrated Tesseract for OCR, and validated real-time processing using camera input. Later iterations developed the ESP32 firmware, WiFi communication, and OLED display routines, followed by MongoDB integration and end-to-end testing. Agile was chosen because it supports frequent testing with real video data, allows adjustments to camera placement and preprocessing, and simplifies integration of distributed components (camera, Python host, ESP32, database).

2.4 Implementation Details

The Python pipeline captures frames from the camera using OpenCV, resizes or formats them as needed, and passes them to a YOLO model for plate detection. For each detected plate, a region of interest (ROI) is extracted and converted to grayscale for OCR. Tesseract is configured with a whitelist of uppercase letters and digits and a page segmentation mode optimized for single plate-like strings. To prevent multiple entries for the same vehicle while it remains in view, a 5-second cooldown is applied between accepted detections. Plates that pass validation are sent via a simple HTTP request to the ESP32, which updates the OLED display, and are logged to MongoDB with timestamps for later analysis.

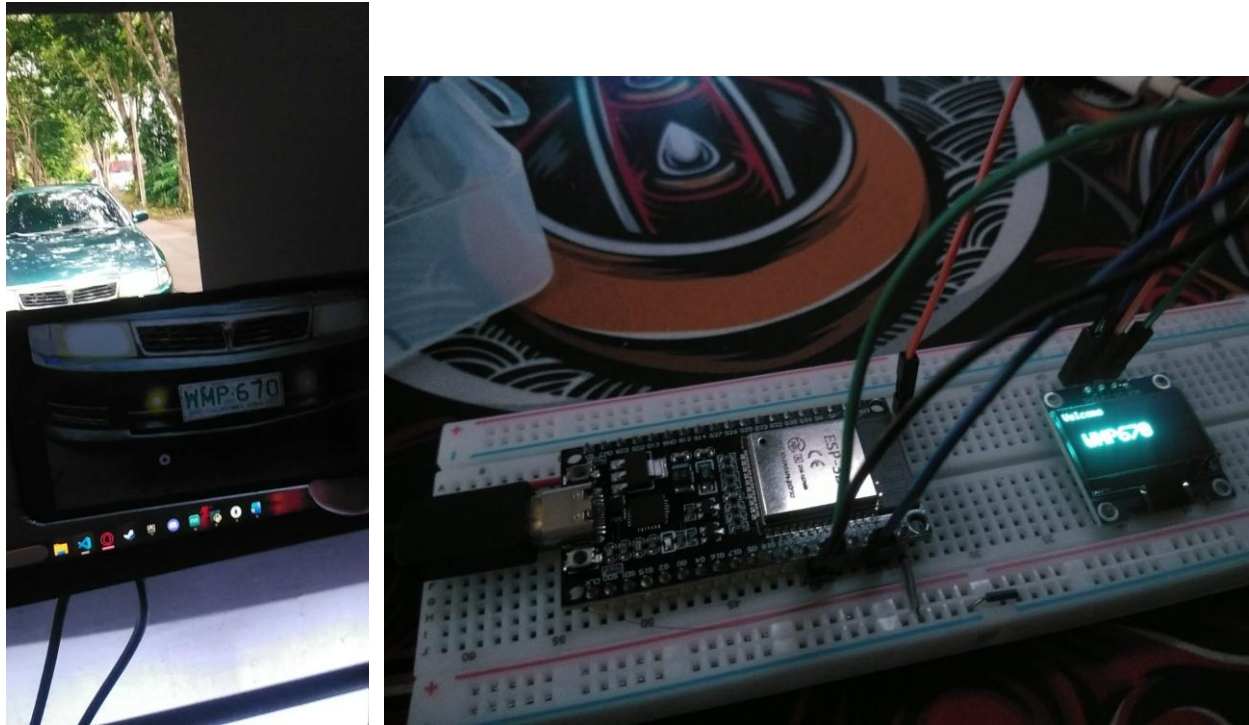
III. Results and Discussion

3.1 System Testing Results

The prototype was evaluated using a fixed camera positioned to detect nameplates. The test scenario included multiple vehicle plates. Captured frames were processed in real time, and outputs were monitored both on the OLED display and in the MongoDB records.



Figure 3: Sample Detection Output



3.2 Cost–Benefit and Operational Impact

The prototype architecture is designed to minimize per-lane cost while enabling automated operation. A typical lane requires one camera, one ESP32 kiosk with display, and shared access to the host server and database. Assuming commodity hardware, the estimated capital cost per lane is on the order of a few hundred to around a thousand US dollars, significantly lower than the long-term cost of staffing a manual toll booth.

Replacing a full-time toll operator with an automated ALPR lane can reduce monthly labor expenses while increasing throughput, because vehicles no longer wait for human interaction to complete payments. This enables parking operators to handle more vehicles with fewer staffed

lanes, or to redirect staff to exception handling and customer support instead of routine toll collection.

3.3 Practical Considerations and Challenges

Network reliability is crucial, as the ESP32 relies on WiFi to receive recognized plate data. During testing, optimizing payload size and ensuring stable local network configuration helped avoid visible delays on the kiosk display. Image quality and camera placement were also important: fine-tuning exposure and angle improved detection performance in both daytime and nighttime conditions. The 10-second cooldown logic effectively eliminated most duplicate entries when vehicles paused in front of the kiosk.

From an operational perspective, ambiguous or low-confidence plates can be routed to a manual review process in a back-office application, ensuring that the system supports robust billing even when some detections are imperfect. MongoDB logs provide the foundation for such workflows as well as for higher-level analytics like peak traffic periods and frequent-visitor identification.

3.4 Limitations and Future Work

The current system is tuned to a specific plate style and region, and performance may drop when confronted with unusual plate formats or heavy weather conditions such as rain and fog. It assumes reliable local connectivity and a stable camera mount. Future work can extend the model with additional training data from multiple regions, integrate vehicle type or color classification, and support dynamic pricing based on occupancy metrics. Further integration with mobile payment platforms and license-plate-based account systems could enable fully touchless parking experiences.

IV. Conclusion

This project implemented and evaluated an Automated License Plate Recognition system intended to automate toll collection in parking lots and reduce congestion at exit lanes. The system combines a camera-based video feed, YOLO-based license plate detection, Tesseract OCR, an ESP32 kiosk with OLED display, and a MongoDB backend, forming a complete pipeline from real-time detection to driver feedback and data logging. Testing in a parking-like environment showed that the system can achieve plate detection and recognition performance suitable for high-throughput scenarios, with sub-second end-to-end response and effective duplicate filtering via a 5-second cooldown.

By reducing reliance on manual toll operators, the system offers potential savings in operational cost and supports higher lane throughput, making it attractive for busy parking facilities. The modular architecture also enables scaling to multiple lanes and integration with external payment and analytics systems. While further work is needed to improve robustness under challenging conditions and to handle diverse plate formats, the results indicate that ALPR-based kiosks are a practical and cost-effective alternative to traditional parking toll booths.